You are a deep learning expert who is good at coding neural networks. You will be given undergraduate level problems to implement layers and neural networks from scratch. You will be given a function definition with a docstring, please fill in the function. Do not make any...

Show thinking

I understand. I am ready to assist. Please provide the skeleton code and the function definition you would like me to implement.

def affine_forward(x, w, b):
"""
Computes the forward pass for an affine (fully-connected) layer.
...

Show thinking

Here is the implemented function.

Python

```python
import numpy as np

def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # TODO: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                 #
    ###########################################################################
    # Reshape x into (N, D). x.shape[0] is N, -1 infers D.
    x_reshaped = x.reshape(x.shape[0], -1)

    # Compute the linear transformation: out = XW + b
    out = x_reshaped.dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b)
    return out, cache
```

Explanation of the Solution

1. **Reshaping:** The input `x` has dimensions $(N, d_1, \ldots, d_k)$. To perform matrix multiplication

with the weights W (which expect a flattened input vector of size D), we flatten all dimensions except the batch size N. Using `x.reshape(x.shape[0], -1)` efficiently transforms the shape to (N, D).

2. **Linear Transformation**: We calculate the dot product of the reshaped input and the weights, then add the bias. The bias b is broadcasted across the batch.

Would you like me to implement the corresponding `affine_backward` function next?

def affine_backward(dout, cache):
"""
Computes the backward pass for an affine layer.

Inputs:...

Show thinking

Here is the implementation for the backward pass of the affine layer.

Python

```python
import numpy as np

def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the affine backward pass.                               #
    ###########################################################################
    # Reshape x to (N, D) to align with the forward pass logic
    x_reshaped = x.reshape(x.shape[0], -1)

    # db: Sum of gradients across the batch dimension (N)
    db = np.sum(dout, axis=0)

    # dw: X^T * dout
    dw = x_reshaped.T.dot(dout)

    # dx: dout * W^T, then reshaped back to original input shape
    dx = dout.dot(w.T).reshape(x.shape)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db
```

Explanation of the Gradients

1. **Bias Gradient (db)**: Since the bias b is added to every example in the batch during the forward pass, the gradient is the sum of the upstream derivatives across the batch dimension (axis 0).

$$db = \sum_{i=1}^{N} dout_i$$

2. **Weight Gradient (dw)**: Derived from the matrix multiplication $Y = XW$. The gradient is the dot product of the transposed input and the upstream derivative.

$$dw = X_{reshaped}^T \cdot dout$$

3. **Input Gradient (dx)**: Derived from the same matrix multiplication. The gradient with respect to the input is the dot product of the upstream derivative and the transposed weights. It must be reshaped back to the original dimensions of x.

$$dx = (dout \cdot W^T)_{\text{reshaped}}$$

Would you like me to move on to the ReLU activation layer next?

def relu_forward(x):
"""
Computes the forward pass for a layer of rectified linear units (ReLUs).
...

Show thinking

Here is the implementation for the ReLU forward pass.

Python

```python
import numpy as np

def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # TODO: Implement the ReLU forward pass.                                  #
    ###########################################################################
    # Element-wise maximum of 0 and x
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache
```

Explanation

- **Operation**: The ReLU function is defined as $f(x) = \max(0, x)$.
- **Implementation**: `np.maximum(0, x)` performs an element-wise comparison between 0 and the input array `x`, effectively setting all negative numbers to zero while keeping positive numbers unchanged.

Would you like me to implement `relu_backward` next?

def relu_backward(dout, cache):
"""

Computes the backward pass for a layer of rectified linear units (ReLUs)....

Show thinking

Here is the implementation for the backward pass of the ReLU layer.

Python

```python
import numpy as np

def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # TODO: Implement the ReLU backward pass.                                 #
    ###########################################################################
    # The gradient is passed through where x > 0, and killed (0) where x <= 0.
    dx = dout * (x > 0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx
```

Explanation

The derivative of the ReLU function $f(x) = \max(0, x)$ is:

- 1 if $x > 0$
- 0 if $x \leq 0$

Therefore, `dx` is simply the upstream derivative `dout` multiplied by a mask that is 1 where the input `x` was positive and 0 otherwise.

Would you like to move on to the `svm_loss` or `softmax_loss` functions next?

Here are some helper functions defined for you that you can use:

```
from deeplearning.layers import *

def affine_relu_forward(x, w, b):
"""
Convenience layer that perorms an affine transform
followed by a ReLU

Inputs:
- x: Input to the affine layer
- w, b: Weights for the affine layer

Returns a tuple of:
- out: Output from the ReLU
- cache: Object to give to the backward pass
"""
a, fc_cache = affine_forward(x, w, b)
out, relu_cache = relu_forward(a)
```

```
cache = (fc_cache, relu_cache)
return out, cache


def affine_relu_backward(dout, cache):
"""
Backward pass for the affine-relu convenience layer
"""
fc_cache, relu_cache = cache
da = relu_backward(dout, relu_cache)
dx, dw, db = affine_backward(da, fc_cache)
return dx, dw, db
```

Keep these in mind for future problems

Show thinking

I have noted the `affine_relu_forward` and `affine_relu_backward` helper functions. I will make sure to utilize these convenience layers in future problems where an affine transform is immediately followed by a ReLU activation.

Please provide the next function definition you would like me to implement.

Here are some loss functions you can use:

```
def svm_loss(x, y):
"""...
```

Show thinking

I have noted the `svm_loss` and `softmax_loss` functions.

I now have the following building blocks:

- **Layers:** `affine_forward`, `affine_backward`, `relu_forward`, `relu_backward`
- **Convenience:** `affine_relu_forward`, `affine_relu_backward`
- **Losses:** `svm_loss`, `softmax_loss`

I am ready to assemble these into a full neural network model. Please provide the class definition and skeleton code for the network you would like me to implement (e.g., `TwoLayerNet`).

Now implement this:

```
class TwoLayerNet(object):
"""
A two-layer fully-connected neural network with ReLU
nonlinearity and
softmax loss that uses a modular layer design. We
assume an input dimension
of D, a hidden dimension of H, and perform classification
over C classes.

The architecure should be affine - relu - affine -
```

```
softmax.

Note that this class does not implement gradient
descent; instead, it
will interact with a separate Solver object that is
responsible for running
optimization.

The learnable parameters of the model are stored in the
dictionary
self.params that maps parameter names to numpy
arrays.
"""

def __init__(self, input_dim=3 * 32 * 32, hidden_dim=100,
num_classes=10,
weight_scale=1e-3, reg=0.0):
"""
Initialize a new network.

Inputs:
- input_dim: An integer giving the size of the input
- hidden_dim: An integer giving the size of the hidden
layer
- num_classes: An integer giving the number of classes
to classify
- dropout: Scalar between 0 and 1 giving dropout
strength.
- weight_scale: Scalar giving the standard deviation for
random
initialization of the weights.
- reg: Scalar giving L2 regularization strength.
"""
self.params = {}
self.reg = reg

############################################
############################################
##
# TODO: Initialize the weights and biases of the two-
layer net. Weights #
# should be initialized from a Gaussian with standard
deviation equal to #
# weight_scale, and biases should be initialized to zero.
All weights and #
# biases should be stored in the dictionary self.params,
with first layer #
# weights and biases using the keys 'W1' and 'b1' and
second layer weights #
# and biases using the keys 'W2' and 'b2'. #
############################################
############################################
##
############################################
############################################
##
# END OF YOUR CODE #
############################################
############################################
##
```

```
def loss(self, X, y=None):
"""
Compute loss and gradient for a minibatch of data.

Inputs:
- X: Array of input data of shape (N, d_1, ..., d_k)
- y: Array of labels, of shape (N,). y[i] gives the label for
X[i].

Returns:
If y is None, then run a test-time forward pass of the
model and return:
- scores: Array of shape (N, C) giving classification
scores, where
scores[i, c] is the classification score for X[i] and class c.

If y is not None, then run a training-time forward and
backward pass and
return a tuple of:
- loss: Scalar value giving the loss
- grads: Dictionary with the same keys as self.params,
mapping parameter
names to gradients of the loss with respect to those
parameters.
"""
scores = None
############################################
############################################
##
# TODO: Implement the forward pass for the two-layer
net, computing the #
# class scores for X and storing them in the scores
variable. #
############################################
############################################
##
############################################
############################################
##
# END OF YOUR CODE #
############################################
############################################
##

# If y is None then we are in test mode so just return
scores
if y is None:
return scores

loss, grads = 0, {}
############################################
############################################
##
# TODO: Implement the backward pass for the two-layer
net. Store the loss #
# in the loss variable and gradients in the grads
dictionary. Compute data #
# loss using softmax, and make sure that grads[k] holds
the gradients for #
# self.params[k]. Don't forget to add L2 regularization on
the weights. #
```

```
# but not the biases. #
# #
# NOTE: To ensure that your implementation matches
ours and you pass the #
# automated tests, make sure that your L2 regularization
includes a factor #
# of 0.5 to simplify the expression for the gradient. #
#####################################
#####################################
##
#####################################
#####################################
##
# END OF YOUR CODE #
#####################################
#####################################
##

return loss, grads
```

Show thinking

Here is the implementation of the `TwoLayerNet` class, utilizing the modular layers and helper functions defined previously.

Python

```python
import numpy as np
from deeplearning.layers import * # Assuming previous functions are in this namespa

class TwoLayerNet(object):
    """
    A two-layer fully-connected neural network with ReLU nonlinearity and
    softmax loss that uses a modular layer design. We assume an input dimension
    of D, a hidden dimension of H, and perform classification over C classes.

    The architecure should be affine - relu - affine - softmax.

    Note that this class does not implement gradient descent; instead, it
    will interact with a separate Solver object that is responsible for running
    optimization.

    The learnable parameters of the model are stored in the dictionary
    self.params that maps parameter names to numpy arrays.
    """

    def __init__(self, input_dim=3 * 32 * 32, hidden_dim=100, num_classes=10,
                 weight_scale=1e-3, reg=0.0):
        """
        Initialize a new network.

        Inputs:
        - input_dim: An integer giving the size of the input
        - hidden_dim: An integer giving the size of the hidden layer
        - num_classes: An integer giving the number of classes to classify
        - dropout: Scalar between 0 and 1 giving dropout strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - reg: Scalar giving L2 regularization strength.
        """
        self.params = {}
        self.reg = reg

        ############################################################################
        # TODO: Initialize the weights and biases of the two-layer net. Weights
        # should be initialized from a Gaussian with standard deviation equal to
        # weight_scale, and biases should be initialized to zero. All weights and
        # biases should be stored in the dictionary self.params, with first layer
        # weights and biases using the keys 'W1' and 'b1' and second layer weights
        # and biases using the keys 'W2' and 'b2'.
        ############################################################################
```

```
        ############################################################################
        self.params['W1'] = weight_scale * np.random.randn(input_dim, hidden_dim)
        self.params['b1'] = np.zeros(hidden_dim)
        self.params['W2'] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params['b2'] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE
        ############################################################################

    def loss(self, X, y=None):
        """
        Compute loss and gradient for a minibatch of data.

        Inputs:
        - X: Array of input data of shape (N, d_1, ..., d_k)
        - y: Array of labels, of shape (N,). y[i] gives the label for X[i].

        Returns:
        If y is None, then run a test-time forward pass of the model and return:
        - scores: Array of shape (N, C) giving classification scores, where
          scores[i, c] is the classification score for X[i] and class c.

        If y is not None, then run a training-time forward and backward pass and
        return a tuple of:
        - loss: Scalar value giving the loss
        - grads: Dictionary with the same keys as self.params, mapping parameter
          names to gradients of the loss with respect to those parameters.
        """
        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the two-layer net, computing the
        # class scores for X and storing them in the scores variable.
        ############################################################################
        W1, b1 = self.params['W1'], self.params['b1']
        W2, b2 = self.params['W2'], self.params['b2']

        # Layer 1: Affine -> ReLU
        out1, cache1 = affine_relu_forward(X, W1, b1)

        # Layer 2: Affine
        scores, cache2 = affine_forward(out1, W2, b2)
        ############################################################################
        #                             END OF YOUR CODE
        ############################################################################

        # If y is None then we are in test mode so just return scores
        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # TODO: Implement the backward pass for the two-layer net. Store the loss
        # in the loss variable and gradients in the grads dictionary. Compute data
        # loss using softmax, and make sure that grads[k] holds the gradients for
        # self.params[k]. Don't forget to add L2 regularization on the weights,
        # but not the biases.
        #
        # NOTE: To ensure that your implementation matches ours and you pass the
        # automated tests, make sure that your L2 regularization includes a factor
        # of 0.5 to simplify the expression for the gradient.
        ############################################################################
        # 1. Compute loss
        data_loss, dscores = softmax_loss(scores, y)
        reg_loss = 0.5 * self.reg * (np.sum(W1**2) + np.sum(W2**2))
        loss = data_loss + reg_loss

        # 2. Compute Gradients
        # Backprop Layer 2 (Affine)
        dx1, dW2, db2 = affine_backward(dscores, cache2)

        # Backprop Layer 1 (Affine -> ReLU)
        dx, dW1, db1 = affine_relu_backward(dx1, cache1)

        # Add regularization to the weight gradients
        dW2 += self.reg * W2
        dW1 += self.reg * W1

        # Store in dictionary
        grads['W2'] = dW2
        grads['b2'] = db2
        grads['W1'] = dW1
        grads['b1'] = db1
```

```
        ##############################################################################
        #                             END OF YOUR CODE                               #
        ##############################################################################

        return loss, grads
```

Here's a solver / optimizer that you can use:

```
import numpy as np
```

from deeplearning import optim...

Show thinking

Here is the code to train the `TwoLayerNet` using the `Solver` class. I have selected hyperparameters that typically achieve >50% accuracy on standard datasets like CIFAR-10.

Python

```python
model = TwoLayerNet(hidden_dim=100, reg=0.5)
solver = None

##############################################################################
# TODO: Use a Solver instance to train a TwoLayerNet that achieves at least  #
# 50% accuracy on the validation set.                                        #
##############################################################################
solver = Solver(model, data,
                update_rule='sgd',
                optim_config={
                  'learning_rate': 1e-3,
                },
                lr_decay=0.95,
                num_epochs=10, batch_size=100,
                print_every=100)
solver.train()
##############################################################################
#                             END OF YOUR CODE                               #
##############################################################################
```

Explanation of Hyperparameters

- **Hidden Dimension (`hidden_dim=100`)**: A size of 100 provides enough capacity for the network to learn features without being too computationally expensive.
- **Regularization (`reg=0.5`)**: A moderate L2 regularization helps prevent overfitting.
- **Learning Rate (`1e-3`)**: This is a standard starting point for SGD.
- **Learning Rate Decay (`0.95`)**: Slightly decaying the learning rate helps the model settle into a minimum as training progresses.
- **Epochs (`10`)**: Ten epochs are generally sufficient for a small network to reach convergence on datasets like CIFAR-10.

Would you like to move on to implementing Multilayer Perceptrons (arbitrary depth) next?

Read through the `FullyConnectedNet` class in the file `deeplearning/classifiers/fc_net.py`.

Implement the initialization, the forward pass, and the backward pass. For the moment don't worry about...

Show thinking

Here is the implementation of the `FullyConnectedNet` class. I have implemented the initialization to support arbitrary depth and filled in the forward and backward passes using the helper functions.

Python

```python
import numpy as np
from deeplearning.layers import *

class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
    ReLU nonlinearities, and a softmax loss function. This will also implement
    dropout and batch normalization as options. For a network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """

    def __init__(self, hidden_dims, input_dim=3 * 32 * 32, num_classes=10,
                 dropout=0, use_batchnorm=False, reg=0.0,
                 weight_scale=1e-2, dtype=np.float32, seed=None):
        """
        Initialize a new FullyConnectedNet.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
        self.dtype = dtype
        self.params = {}

        ############################################################################
        # TODO: Initialize the parameters of the network, storing all values in
        # the self.params dictionary. Store weights and biases for the first layer
        # in W1 and b1; for the second layer use W2 and b2, etc. Weights should be
        # initialized from a normal distribution with standard deviation equal to
        # weight_scale and biases should be initialized to zero.
        ############################################################################
        # Create a list of all layer dimensions: [input, hidden1, hidden2, ..., ou
        layer_dims = [input_dim] + hidden_dims + [num_classes]

        for i in range(self.num_layers):
            # i corresponds to the layer index starting from 0 (for the list),
            # but params are 1-indexed (W1, W2, etc.)
            idx = i + 1

            self.params[f'W{idx}'] = weight_scale * np.random.randn(layer_dims[i],
            self.params[f'b{idx}'] = np.zeros(layer_dims[i+1])
        ############################################################################
        #                             END OF YOUR CODE
        ############################################################################

        # When using dropout we need to pass a dropout_param dictionary to each
        # dropout layer so that the layer knows the dropout probability and the mo
        # (train / test). You can pass the same dropout_param to each dropout laye
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {'mode': 'train', 'p': dropout}
            if seed is not None:
                self.dropout_param['seed'] = seed

        # With batch normalization we need to keep track of running means and
        # variances, so we need to pass a special bn_param object to each batch
        # normalization layer. You should pass self.bn_params[0] to the forward pa
        # of the first batch normalization layer, self.bn_params[1] to the forward
        # pass of the second batch normalization layer, etc.
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{'mode': 'train'} for i in range(self.num_layers - 1

        # Cast all parameters to the correct datatype
```

```
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.

        Input / output: Same as TwoLayerNet above.
        """
        X = X.astype(self.dtype)
        mode = 'test' if y is None else 'train'

        # Set train/test mode for batchnorm params and dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param['mode'] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param[mode] = mode

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the fully-connected net, computing
        # the class scores for X and storing them in the scores variable.
        ############################################################################
        caches = []
        out = X

        # 1. Forward pass for hidden layers (Affine -> ReLU)
        # We loop from layer 1 up to num_layers - 1
        for i in range(self.num_layers - 1):
            idx = i + 1
            W = self.params[f'W{idx}']
            b = self.params[f'b{idx}']
            out, cache = affine_relu_forward(out, W, b)
            caches.append(cache)

        # 2. Forward pass for the last layer (Affine only, no ReLU)
        idx = self.num_layers
        W_last = self.params[f'W{idx}']
        b_last = self.params[f'b{idx}']
        scores, cache = affine_forward(out, W_last, b_last)
        caches.append(cache)
        ############################################################################
        #                             END OF YOUR CODE
        ############################################################################

        # If test mode return early
        if mode == 'test':
            return scores

        loss, grads = 0.0, {}
        ############################################################################
        # TODO: Implement the backward pass for the fully-connected net. Store the
        # loss in the loss variable and gradients in the grads dictionary. Compute
        # data loss using softmax, and make sure that grads[k] holds the gradients
        # for self.params[k]. Don't forget to add L2 regularization on the
        # weights, but not the biases.
        ############################################################################
        # 1. Compute Loss (Softmax + L2 Reg)
        data_loss, dscores = softmax_loss(scores, y)
        reg_loss = 0
        for i in range(self.num_layers):
            W = self.params[f'W{i+1}']
            reg_loss += 0.5 * self.reg * np.sum(W * W)

        loss = data_loss + reg_loss

        # 2. Backward pass
        # Backprop the last layer (Affine)
        dout, dW, db = affine_backward(dscores, caches.pop())
        idx = self.num_layers
        grads[f'W{idx}'] = dW + self.reg * self.params[f'W{idx}']
        grads[f'b{idx}'] = db

        # Backprop the hidden layers (Affine -> ReLU)
        # Loop backwards from num_layers - 2 down to 0
        for i in range(self.num_layers - 2, -1, -1):
            idx = i + 1
            dout, dW, db = affine_relu_backward(dout, caches.pop())
            grads[f'W{idx}'] = dW + self.reg * self.params[f'W{idx}']
```

```
        grads[f'b{idx}'] = db
    ############################################################################
    #                             END OF YOUR CODE
    ############################################################################

    return loss, grads
```

Implementation Notes

1. **Dynamic Initialization**: The code generates `W` and `b` keys dynamically based on the length of `hidden_dims` + 1 (for the output layer).
2. **Caches**: During the forward pass, I store the cache for every layer in a list `caches`. In the backward pass, I `pop()` these caches to retrieve them in reverse order (LIFO), which aligns perfectly with backpropagation.
3. **Regularization**: The L2 regularization term $0.5 \cdot \lambda \cdot W^2$ is added to the loss, and the gradient term $\lambda \cdot W$ is added to the weight gradients.

Would you like to proceed with implementing Batch Normalization or Dropout next?

As another sanity check, make sure you can overfit a small dataset of 50 images. First we will try a three-layer network with 100 units in each hidden layer. You will need to tweak the learning rate and initialization scale, but you should be able to overfit and achieve 100%...

Show thinking

Here are the hyperparameters that should allow you to overfit the small dataset of 50 images within 20 epochs.

Python

```
############################################################################
# TODO: Use a three-layer Net to overfit 50 training examples.             #
############################################################################
weight_scale = 1e-2   # Standard initialization scale
learning_rate = 1e-2  # Aggressive enough for SGD to converge in 20 epochs
############################################################################
#                             END OF YOUR CODE                             #
############################################################################
```

Explanation

- `weight_scale = 1e-2`: This initializes the weights with a standard deviation of 0.01. This is generally small enough to prevent immediate activation saturation or exploding gradients, but large enough to propagate the signal effectively through a 3-layer network.
- `learning_rate = 1e-2`: Since we are using standard Stochastic Gradient Descent (SGD) without momentum (the default in your solver setup), we need a relatively high learning rate to traverse the loss landscape quickly enough to reach zero loss within just 20 epochs.

Would you like to move on to implementing Batch Normalization or Dropout next?

Now try to use a five-layer network with 100 units on each layer to overfit 50 training examples. Again you will have to adjust the learning rate and weight initialization, but you should be able to achieve 100% training accuracy within 20 epochs....

Show thinking

Show thinking

For a 5-layer network, initialization becomes critical. If the weights are too small, the gradients will vanish (go to zero) as they backpropagate through the layers. If they are too large, the activations will saturate or explode.

For this specific "overfit small data" task without Batch Normalization, using a slightly larger weight scale than the 3-layer case usually helps propagate the signal, paired with a moderate learning rate.

Python

```
################################################################################
# TODO: Tune these parameters to get 100% train accuracy within 20 epochs. #
################################################################################
learning_rate = 2e-3
weight_scale = 1e-1
################################################################################
#                              END OF YOUR CODE                                #
################################################################################
```

Ask Gemini

Tools

Thinking

Gemini can make mistakes, so double-check it